

CS6888 Software Analysis and its Applications

HW4

Sravanth Chowdary Potluri
und5uv@virginia.edu
University Of Virginia

I. ANALYSIS OF TCAS5.C

This section presents the results of running `cppcheck` on `tcas5.c`, a modified version of the Traffic Collision Avoidance System, in two configurations: a standard run (`cppcheck --output-file=<file> tcas5.c`) and a bug-hunting run (`cppcheck --output-file=<file> --bug-hunting tcas5.c`), as specified in Assignment 4, Section 4.1.2. The objective, per Section 4.2(1), is to catalog all faults reported by both runs in a table and classify each as a certain fault, potential fault, or false positive, with justifications derived from code analysis.

A. Fault Table and Classification

Table I compiles the faults identified by `cppcheck` across both runs. The standard run reported two primary errors, with accompanying notes providing context, while the bug-hunting run flagged three distinct errors at a single line, reflecting its more aggressive analysis mode. Each fault is analyzed below, with classifications based on the code's logic and potential runtime behavior.

B. Analysis

The analysis of each fault leverages the provided `tcas5.c` source code and `cppcheck` outputs to determine its validity and severity. The standard run focuses on definitive errors, while the bug-hunting run explores more speculative issues, aligning with their respective design goals.

1) *Array Index Out of Bounds (Line 75, Standard Run)*: In the standard run, `cppcheck` flags an array access at line 75 within the `ALIM()` function: `alim = Positive_RA_Alt_Thresh[4]`. The array `Positive_RA_Alt_Thresh` is defined with four elements (indices 0 to 3), initialized to 400, 500, 640, and 740 in `initialize()`. Accessing index 4, as occurs in the case 4 branch of the switch statement, exceeds these bounds, resulting in undefined behavior. This fault is classified as a *certain fault* because it is a hard-coded error that will invariably occur when `Alt_Layer_Value` equals 4, a value explicitly handled in the switch without bounds checking, leading to memory corruption or a crash.

2) *Signed Integer Overflow (Line 61, Standard Run)*: The standard run also identifies a signed integer overflow at line 61 in `ALIM()`: `alim = Local_Positive_RA_Alt_Thresh * altitude`. Accompanying notes clarify that

`Local_Positive_RA_Alt_Thresh` is assigned 740 at line 60, and `altitude` is set to 1,000,000,000 when `Climb_Inhibit` is true. Multiplying these yields 740,000,000,000, which vastly exceeds the 32-bit signed integer maximum of 2,147,483,647. This overflow introduces undefined behavior, such as wraparound or truncation, compromising the function's return value. Classified as a *certain fault*, this issue is guaranteed to manifest whenever `Climb_Inhibit` is true, given the fixed, excessively large `altitude` value, making it a critical defect in this perturbed version of the code.

3) *Array Index Out of Bounds (Alt_Layer_Value ≥ 4, Line 93, Bug-Hunting Run)*: Switching to the bug-hunting run, `cppcheck` reports a potential array index out-of-bounds error at line 93 in `Non_Crossing_Biased_Climb()`: `Inhibit_Biased_Climb() / Positive_RA_Alt_Thresh[Alt_Layer_Value]`. The tool notes it cannot determine if `Alt_Layer_Value` is less than 4. Since `Positive_RA_Alt_Thresh` has valid indices 0 to 3, an `Alt_Layer_Value` of 4 or greater—possible via command-line input (`argv[7]`) without validation—would access beyond the array's bounds. This is a *potential fault* because, unlike line 75, the error depends on runtime input rather than a fixed code path, requiring specific conditions (e.g., `Alt_Layer_Value = 4`) to trigger undefined behavior.

4) *Array Index Out of Bounds (Alt_Layer_Value < 0, Line 93, Bug-Hunting Run)*: A second fault at line 93 from the bug-hunting run warns that `Alt_Layer_Value` could be negative, as `cppcheck` cannot confirm it is non-negative. In the same expression, a negative index would result in an invalid memory access, potentially causing a segmentation fault. The code accepts `Alt_Layer_Value` from `argv[7]` via `atoi()`, which can produce negative values if a negative string (e.g., "-1") is provided, and no lower-bound check exists. This is classified as a *potential fault*, as it hinges on user input rather than an inherent code flaw, but it underscores a lack of input sanitization that could destabilize the program.

5) *Potential Division by Zero (Line 93, Bug-Hunting Run)*: The final bug-hunting fault at line 93 flags a possible division by zero in `Inhibit_Biased_Climb() / Positive_RA_Alt_Thresh[Alt_Layer_Value]`. The array's initialized values (400, 500, 640, 740) are all positive, so within bounds, the denominator is never zero. However, if `Alt_Layer_Value` is out of bounds (e.g.,

≥ 4 or < 0), the access could theoretically hit uninitialized memory, which might be zero, though this is unlikely given the static allocation. Classified as a *potential fault*, this issue is speculative and less immediate than the array bounds errors, as the initialized array mitigates the risk, but it reflects cppcheck’s slightly speculative analysis in bug-hunting mode.

II. ANALYSIS OF TCC

A. cppcheck and infer Results

Table II enumerates the faults detected by cppcheck and infer when applied to the tcc-0.9.27 codebase using the configurations specified in Sections 4.1.1 and 4.1.3. The table lists each fault with its corresponding file name, line number, and fault type, sorted alphabetically by file name and then by ascending line number, excluding infer’s UNINITIALIZED_VALUE reports and cppcheck’s notes to focus on actionable errors such as null pointer dereferences, array bounds violations, and dead stores.

B. Trace Analysis

Using the HTML report from Subsubsection 4.1.3, we evaluate four issue traces identified by infer to determine whether each is a *certain fault*, *potential fault*, or *false positive*. The analysis relies on the code snippets in the traces, with additional context from the enclosing function where referenced, and provides an explanation for each classification.

a) *Issue 1: libtcc.c:174 – Null Dereference of pointer p*: Classification: Potential Fault

Explanation: The trace shows `p` assigned by `strchr(name, 0)` at line 173, which seeks the null terminator in `name` and returns a pointer to it or `NULL` if the character is not found. At line 174, the loop condition `!IS_DIRSEP(p[-1])` dereferences `p[-1]` without checking if `p` is `NULL`, which could occur if `name` is invalid or null itself. Although `p > name` guards against underflow, it does not prevent a null dereference if `strchr` fails, making this a potential fault under rare input conditions.

b) *Issue 8: libtcc.c:441 – Uninitialized Value c*: Classification: Potential/Certain Fault

Explanation: The trace at line 441 shows a loop where `c` is assigned via `c = *p` in the condition, but `c` is not initialized before the loop begins. The comma expression evaluates `c = *p` first, then uses `c` in subsequent checks, but if `p` points to invalid memory, the initial read is undefined, and no prior initialization of `c` is evident in the trace. This makes infer’s uninitialized value report a Potential fault, as `c`’s value is doesn’t seem to be initialized before its first assignment and use.

c) *Issue 51: tcclself.c:2604 – Null Dereference of pointer data*: Classification: Potential/Certain Fault

Explanation: The trace shows `data` allocated via `tcc_malloc(size)` at line 2601, which calls `malloc(size)` (line 207) and may return `NULL` if allocation fails. Although the trace assumes success (false branch at line 208), no null check precedes

`get_be32(data)` at line 2604, which dereferences `data`. This is a potential fault, as a memory allocation failure in practice could lead to a null dereference. However, this fault might not be triggered at all if the `tcc_error` raised stops the execution of the program, but it is still a potential fault because the code does not check for null after the allocation and the null dereference could happen if the allocation fails and a null pointer is passed to `get_be32(data)`.

d) *Issue 62: tccgen.c:4100 – Null Dereference of pointer type->ref*: Classification: False Positive

Explanation: In `parse_btype`, `type->ref` is set to `NULL` at line 4012, and at line 4100, `parse_btype_qualify(type, VT_CONSTANT)` is called. Within `parse_btype_qualify`, the loop at line 3991 (`while (type->t & VT_ARRAY)`) is false, skipping any dereference of `type->ref`, so no null dereference occurs in this path, rendering infer’s warning a false positive.

C. Likelihood and Trends of Reported Faults

Using the data from Table II and the trace analyses, we assess the likelihood of the reported items being actual faults and identify trends in fault types across the tcc-0.9.27 codebase. Table II lists 31 faults from cppcheck and infer, spanning null pointer dereferences, array index out-of-bounds errors, shift overflows, dead stores, and a memory leak. The trace analyses of four specific infer issues reveal a mixed picture: Issue 62 (`tccgen.c:4100`) is a false positive due to a safe execution path avoiding a null dereference, while Issues 1 (`libtcc.c:174`), 8 (`libtcc.c:441`), and 51 (`tcclself.c:2604`) are potential faults contingent on rare conditions like invalid inputs or memory allocation failures. This suggests that while many reported items are plausible, their fault likelihood varies significantly based on context.

The prevalence of potential/certain faults in the traces—three out of four—indicates that infer often flags issues requiring specific runtime scenarios to manifest, such as a null `name` in `tcc_basename` (Issue 1) or a failed `malloc` in `tcc_load_alacarte` (Issue 51). For Issue 8, the loop’s comma operator ensures `c` is assigned before use, yet the lack of explicit initialization outside the loop raises a potential fault if `p` is invalid, though this is mitigated by typical string handling assumptions. Extrapolating to the table, null dereferences (e.g., nine from infer, four from cppcheck) and array bounds errors (e.g., `c67-gen.c:1602, 1773`) dominate, comprising over 50% of reports, reflecting tcc’s heavy reliance on pointers and arrays in C. Dead stores (12 instances, all from infer) suggest optimization issues rather than critical bugs, lowering their fault likelihood, while rarer types like `shiftTooManyBits` (e.g., `arm64-link.c:184, 188`) and `MEMORY_LEAK` (e.g., `libtcc.c:1655`) indicate specific architectural or resource management flaws with higher severity when triggered.

Overall, approximately one-third of the reported items are likely certain faults—such as definitive array overruns or shift

overflows—due to their unambiguous violation of C semantics, while the majority are potential faults dependent on execution paths or inputs, and a minority, like Issue 62, are false positives from over-conservative analysis. The trend toward pointer-related issues (null dereferences, arithmetic errors) underscores `tcc`’s vulnerability in memory management, a common challenge in low-level C code, whereas dead stores highlight static analysis sensitivity to unused assignments that may not impact runtime behavior.

III. LIMITATIONS

This section examines the limitations of `cppcheck` and `infer` based on their performance on `tcas5.c` and `tcc-0.9.27`, focusing on the flags specified in Sections 4.1.1 to 4.1.3, and assesses whether the observed issues stem from fundamental constraints of static analysis or could be mitigated with available options. The analysis draws from the fault classifications and trends identified earlier, highlighting false positives and incomplete contextual understanding as recurring challenges.

For `cppcheck`, a notable limitation is its propensity to generate false positives, as seen in the standard run on `tcas5.c`, where conservative checks flagged potential issues like unused variables (not reported here due to filtering) that may not impact functionality. The tool’s standard mode (`cppcheck --output-file=<file>`) prioritizes broad coverage, while the bug-hunting mode (`--bug-hunting`) amplifies this by speculatively identifying risks like the division-by-zero at `tcas5.c:93`, which is unlikely given initialized array values. This over-sensitivity is partly a fundamental limitation of static analysis, which must err on caution without runtime data, but could be refined using flags like `--inconclusive` to filter less certain warnings or `--suppress` to exclude specific fault types (e.g., `integerOverflow` was suppressed for `tcc`). However, suppressing too aggressively risks missing true positives, as with the certain fault at `tccpe.c:1005` (null pointer arithmetic), illustrating a trade-off inherent to the approach.

`infer` exhibits similar challenges with false positives, evident in the trace analyses of `tcc`. For instance, Issue 1 (`libtcc.c:174`) was incorrectly flagged as a null dereference despite `strchr(name, 0)` always returning a valid pointer to a string’s null terminator, and Issue 62 (`tccgen.c:4100`) was a false positive due to a safe execution path avoiding dereference. These errors stem from a fundamental limitation: `infer`’s interprocedural analysis, run with `infer run -j 1 --reactive --make`, lacks full runtime context and over-approximates potential issues. The `--reactive` flag, enabling incremental analysis, may exacerbate this by limiting re-evaluation of prior assumptions, but options like `--bi-abduction` (enhancing pointer reasoning) or `--no-filtering` (retaining more reports) could reduce false positives, though at the cost of increased noise. Issue 8 (`libtcc.c:441`), initially misclassified as a certain fault, is actually only a potential fault since `c` might actually be initialized before use, highlighting `infer`’s difficulty with

complex control flow—a limitation partially addressable with finer-grained configuration.

Both tools struggle with incomplete contextual understanding, a core constraint of static analysis. For `tcas5.c`, `cppcheck`’s potential faults at line 93 (array bounds and division-by-zero) depend on runtime inputs like `Alt_Layer_Value`, which static tools cannot fully resolve without execution. Similarly, `infer`’s potential fault at `tccelf.c:2604` (Issue 51) assumes a null data from `malloc` failure, but cannot confirm its likelihood without dynamic behavior, despite a subsequent error check mitigating impact. This incomplete picture is fundamental, as static analysis abstracts away runtime states, though `cppcheck`’s `--enable=all` or `infer`’s `--developer-mode` could provide deeper insights at the expense of scalability.

In summary, false positives and partial context are inherent to the static analysis paradigm, reflecting the tools’ inability to simulate all execution paths precisely. While flags like `--inconclusive` for `cppcheck` or `--bi-abduction` for `infer` offer partial remedies by tuning precision, they cannot fully overcome the absence of runtime data, underscoring a fundamental trade-off between soundness and practicality in these tools’ designs.

IV. SUMMARY

This section summarizes the performance of `cppcheck` and `infer` across `tcas5.c` and `tcc-0.9.27`, evaluating their effectiveness in identifying issues, the severity of findings, and the presence of false positives and negatives. `cppcheck` detected five issues in `tcas5.c` across standard and bug-hunting runs, with two certain faults (array overrun at line 75, integer overflow at line 61) of high severity due to guaranteed undefined behavior, and three potential faults (line 93) of moderate severity dependent on input conditions. For `tcc`, it reported 10-15 faults excluding notes, including severe issues like array bounds errors (`c67-gen.c:1602, 1773`) and shift overflows (`arm64-link.c:184, 188`), alongside less critical null pointer warnings (`x86_64-gen.c:336`). No false positives were evident in `tcas5.c`, though its conservative approach likely misses subtle runtime-dependent faults (false negatives), such as uncaught edge cases in `tcc`.

`infer` identified 66 issues in `tcc`, with 22 actionable faults listed in Table II after exclusions (nine null dereferences, 12 dead stores, one memory leak). The trace analysis revealed one false positive (Issue 62, `tccgen.c:4100`) and three potential faults (Issues 1, 8, 51), with severity ranging from moderate (e.g., null dereference at `tccelf.c:2604`) to low (e.g., dead stores like `x86_64-gen.c:1743`). potential false negatives—such as missed complex pointer interactions beyond the traces—may exist due to its focus on specific patterns like null dereferences. Overall, `infer` excels at depth, catching nuanced issues like memory leaks (`libtcc.c:1655`), but its 14% actionable fault rate (9/66 excluding dead stores) suggests a trade-off between sensitivity and specificity, as many flagged items are less critical or speculative.

Both tools balance strengths and weaknesses: `cppcheck` offers broad, scalable coverage with fewer false positives in `tcas5.c` (0/5) but limited depth, potentially under-reporting faults in `tcc`, while `infer` provides detailed traces with higher sensitivity (66 issues) at the cost of increased false positives (e.g., 1/4 traced issues). Severity aligns with fault type—array and pointer errors pose high risks, dead stores lower—yet runtime-dependent potential faults complicate assessment without execution. False negatives remain unquantified but likely, given static analysis’s inherent contextual limits, making these tools complementary for robust code verification.

This homework and report are completed with the help of resources provided along with the homework. This report also makes use of Grok-3 to refine the content.

TABLE I
FAULTS REPORTED BY CPPCHECK ON TCAS5.C

Run Type	File	Line	Fault Type	Classification
Standard	tcas5.c	75	Array Index Out of Bounds	Certain Fault
Standard	tcas5.c	61	Signed Integer Overflow	Certain Fault
Bug-Hunting	tcas5.c	93	Array Index Out of Bounds (≥ 4)	Potential Fault
Bug-Hunting	tcas5.c	93	Array Index Out of Bounds (< 0)	Potential Fault
Bug-Hunting	tcas5.c	93	Potential Division by Zero	Potential Fault

TABLE II
FAULTS REPORTED BY CPPCHECK AND INFER ON TCC

File Name	Line	Fault Type
arm64-link.c	184	shiftTooManyBits
arm64-link.c	188	shiftTooManyBits
c67-gen.c	1602	arrayIndexOutOfBounds
c67-gen.c	1773	arrayIndexOutOfBounds
c67-gen.c	2113	uninitvar
libtcc.c	174	NULL_DEREFERENCE
libtcc.c	186	NULL_DEREFERENCE
libtcc.c	217	NULL_DEREFERENCE
libtcc.c	234	NULL_DEREFERENCE
libtcc.c	1655	MEMORY_LEAK
tcelf.c	2604	NULL_DEREFERENCE
tcelf.c	2604	NULL_DEREFERENCE
tcgen.c	4100	NULL_DEREFERENCE
tcgen.c	4108	NULL_DEREFERENCE
tcpe.c	1004	nullPointer
tcpe.c	1004	nullPointer
tcpe.c	1005	nullPointerArithmetic
tcpp.c	732	DEAD_STORE
tcpp.c	737	DEAD_STORE
tcpp.c	2382	DEAD_STORE
tcpp.c	2387	DEAD_STORE
tcpp.c	2439	DEAD_STORE
tcpp.c	2443	DEAD_STORE
tcpp.c	3107	NULL_DEREFERENCE
x86_64-gen.c	336	nullPointer
x86_64-gen.c	336	nullPointer
x86_64-gen.c	1743	DEAD_STORE
x86_64-gen.c	1845	DEAD_STORE
x86_64-gen.c	1937	DEAD_STORE
x86_64-gen.c	1958	DEAD_STORE
x86_64-gen.c	1959	DEAD_STORE